

features, especially hardware features like the camera, GPS, etc. With the ever evolving mobile platforms and hybrid development frameworks, performance is no longer a concern.



Figure 5: Sample folder structure

Coming to accessing device-specific features, there are various CPTs that have JavaScript wrappers, which expose such functionality so that one can access the hardware of the device using JavaScript code. One such CPT is PhoneGap (now known as Cordova). It does two major things - it helps to package the Web page code and expose it as a native application, and it also provides JavaScript wrappers that offer access to the device hardware (Figure 4). These wrappers have a pretty simple underlying concept. Let us consider the example of Android. If I need to access the camera of an Android device, I can use the *Camera* API. Now, to write a wrapper, one needs to write a library that inherently calls the *Camera* API. There are a lot of ways to achieve this. The wrappers are customised, the webview is customised, but internally, the method used is on lines similar to what's been discussed here.

We will now look at a way in which one can create a hybrid app. I will not cover the code because, essentially, it is a normal Web page. Instead, I will focus on how to package it. The most important thing here is to ensure that your Web application follows SPA (Single Page Architecture). One can have multiple and external HTML5 pages, but that is generally not how hybrid apps are designed. The easiest way to package a Web app as a hybrid app is to use *build.phonegap.com*. For instance, let us consider the folder structure shown in Figure 5 (I've used jQuery Mobile, which is a popular framework for making hybrid mobile apps and mobile Web apps). Now, we need to zip this into a single zip file. Then, this file needs to be uploaded on *build.phonegap.com*. There is a lot more involved behind the scenes, such as the config.xml, but for now, we can skip that.. Once the zip is uploaded, the cloud platform will automatically generate a hybrid app for us, for this particular build (see Figure 6—I've used the Ionic framework, another hybrid development framework based on AngularJS).

Experienced developers of native apps might prefer to use Eclipse and the Android SDK as the hybrid way might seem a bit complex to them. They can directly create a sample Android app, and have a single Activity, for which they can define a webview control and then perform the HTML5 magic there. Those interested in cross-platform development can definitely jump into

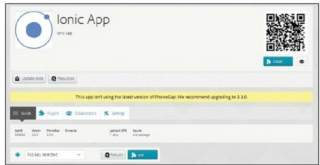


Figure 6: PhoneGap build

hybrid development. For such developers, PhoneGap is a nice place to begin, and just reading a sample of what's available there (<http://phonegap.com/2012/05/02/phonegap-explained-visually/>) can help a lot in understanding PhoneGap and how it works. **END**

By: Manit Singh Kalsi

The author works as a mobile evangelist in the Mobility Centre of Excellence at Genpact Headstrong Capital Markets. He is a Java and JavaScript developer, who enjoys exploring new technologies and frameworks. When not coding, he is either strumming his guitar or playing video games. Follow him at manitsinghkalsi.

PACK WEB HOSTING
Time to go PRO now

www.packwebhosting.com
0-98769-44977
support@packwebhosting.com

A Leading Web & Email Hosting Provider
Specialists in

Hosting Sites

built with OpenSource Technologies

WordPress
 Magento
 Joomla
 Drupal

ProX Plans
Have a High Traffic Website?
Considering VPS/Server?
Visit prox.packwebhosting.com

- cPanel Hosting
- One Click Installation
- Solid Support
- Multiple Hosting Plans

Why Us?

- 4000+ Hosting
- 2000+ Clients
- 6500+ Domains
- 11+ Years Experience

Trust Us. Trust our Ability.